

Computabilità e Complessità

Damiano Trovato

Secondo semestre, anno 2025/26

Indice

0	Introduzione alla computabilità	3
0.1	I personaggi della computabilità	3
0.2	Cos'è un algoritmo?	4
0.2.1	Risolvere un algoritmo	4
0.2.2	Congettura di Church-Turing	4
1	Funzioni computabili e URM	5
1.1	Unlimited Register Machine	5
1.1.1	Configurazione istantanea	5
1.1.2	Istruzioni degli URM-programmi	5
1.2	Definizione di computazione	6
1.3	Funzione parziale	6
1.3.1	Funzioni URM-calcolabili	7
1.4	Esercizi	7
1.4.1	Programmi URM	7
1.4.2	Dimostrazioni	8
1.5	Correttezza dei programmi	9
1.5.1	Esempio di specifiche	9
1.5.2	Verifica della correttezza parziale dei programmi di Floyd/Hoare	9
1.5.3	Verifica della correttezza totale dei programmi	10
1.5.4	Sulla verifica della correttezza	10
1.6	Predicati	10
1.6.1	Predicati decidibili	10
1.7	Calcolabilità su altri domini	10
2	Creare funzioni computabili	11
2.1	Condizioni di esistenza di programmi	11
2.1.1	Operazioni su cui le funzioni calcolabili sono chiuse	11
3	Composizione di programmi	13
3.1	Come concatenare programmi	13
3.1.1	Concatenazione di programmi	13
3.1.2	Problema della concatenazione	13
3.2	Composizione di funzioni (o sostituzione)	14
3.3	Sostituzioni di variabili	14
4	Ricorsione	15
4.1	Ricorsione primitiva	15
4.1.1	Teorema sull'esistenza e unicità	15
4.1.2	Esempi di funzioni calcolabili con ricorsione primitive	16
4.1.3	Teorema sulla calcolabilità della ricorsione primitiva	16
4.2	Funzioni primitive ricorsive	16
4.2.1	Insieme $\mathcal{PR}$	16
5	Alcuni predicati decidibili e funzioni calcolabili	17
5.1	Funzioni definite per casi	17
5.2	Algebra della decidibilità	17
5.3	Sommatorie e produttorie limitate	18
5.3.1	Corollario	18
5.4	Minimalizzazione limitata	18

5.4.1	Corollario - minimalizzazione limitata da una funzione	19
5.4.2	Corollario - minimalizzazione di predicati decidibili	19
5.5	Quantificatori limitati	19
5.6	Codifiche per tuple	19
5.6.1	Codifiche per coppie di numeri	19
5.6.2	Codifiche per sequenze di numeri	19
6	Minimalizzazione	21
6.1	Operazione di minimalizzazione	21
6.1.1	Dimostrazione della calcolabilità	21
6.1.2	Corollario sulla minimalizzazione dei predicati decidibili	22
6.1.3	Funzioni parziali da funzioni totali	22
6.2	Funzioni ricorsive parziali	22
7	Completezza delle funzioni ricorsive parziali	23
7.1	Funzioni ricorsive parziali	23
7.2	Funzione di Ackermann	23
7.2.1	Dimostrazione calcolabilità di $\lambda y.\psi_x(y)$	24
7.2.2	Calcolabilità della funzione di Ackermann	24
8	Loop Program	26
8.1	Istruzioni dei loop program	26
8.1.1	Mappare Loop program in un programma URM	26
8.2	Gerarchia dei Loop program	26
8.2.1	Gerarchia delle <i>funzioni</i> calcolate dai Loop program	26
8.3	Funzioni primitive ricorsive e i Loop Program	27
9	Altri approcci alla computabilità e tesi di Church	28
9.1	Altri approcci alla computabilità	28
9.2	Funzioni ricorsive parziali (Godel - Kleene)	28
9.2.1	Tutte le funzioni ricorsive parziali sono calcolabili	28
9.3	Macchine di Turing	29
9.3.1	Funzioni Turing calcolabili	29
9.3.2	Equivalenza tra URM e TM	29
9.4	Tesi di Church-Turing	30
9.4.1	La potenza della tesi di Church-Turing	30
10	Enumerazione delle funzioni calcolabili	31
10.1	Enumerazione	31
10.1.1	Alcuni risultati tecnici	31
10.2	Enumerabilità delle funzioni calcolabili	32
10.2.1	Enumerabilità delle istruzioni	32
10.2.2	Enumerabilità dei programmi URM	32
10.2.3	Numero di Gödel	32
10.3	Enumerabilità delle funzioni	33
10.4	Diagonalizzazione di Cantor	33
10.4.1	Teorema sull'esistenza di una funzione totale, unitaria e non calcolabile	33
11	Decidibilità	34
11.1	Teorema <i>s-m-n</i> o di parametrizzazione	34
12	Programmi universali	35
12.1	Programma universale	35
12.1.1	Calcolabilità della funzione universale	35
12.2	Esistenza di funzione <i>totale</i> calcolabile non primitiva ricorsiva	35
12.3	Operazioni effettiva su funzioni calcolabili	35
12.3.1	Esempio 1	35
12.3.2	Esempio 2	35
13	???	37
13.1	Problemi indecidibili della teoria della calcolabilità	37
13.1.1	Halting problem	37
13.2	Sinonimi	38

Capitolo 0

Introduzione alla computabilità

0.1 I personaggi della computabilità

Gottfried Wilhelm Von Leibniz Tra gli inizi del 600 e la fine del 700: un grande ottimismo, che pensava il mondo fosse stato sviluppato nel modo migliore. In un'epoca senza computer e tecnologia tale per svilupparli, definisce un linguaggio chiamato **Caratteristica Universale**. Si pongono le basi per la **logica del primo ordine**, il **calcolo deduttivo**. Ogni disputa si sarebbe dovuta risolvere con un calcolo! Ricordiamo il contributo di Leibniz all'analisi matematica con il concetto di **derivata** e la notazione che usiamo oggi: queste notazioni ammettevano delle automazioni! Contribuì anche alla creazione di un meccanismo, noto come **ruota di Leibniz**, che estendeva l'addizionatrice di Pascal con le moltiplicazioni. Di fronte al tentativo di ricavare l'assioma delle parallele, si inizia a definire il concetto di **consistenza delle teorie**. In generale, fu il primo a mettere le basi per un **calcolo universale**.

George Boole Algebrizza la logica proposizionale, ridefinendo il ragionamento usando connettivi logici ($\wedge, \vee, \neg$).

Gottlob Frege Definito spesso come il padre della logica matematica formale. La logica di Frege usava anche molti elementi di teoria degli insiemi.

William Russell Uno dei paradossi posti a Frege, e probabilmente uno dei paradossi più grandi della storia della logica. Sia U l'insieme di tutti gli insiemi che non contengono se stessi. U appartiene a se stesso?

$$U = \{x : x \notin x\}; \quad U \in U?$$

Questo mette in crisi un'assioma della logica di Frege, noto come **assioma di appartenenza**, fin troppo potente, tale da causare il paradosso

$$U \in U \implies U \notin U, \quad U \notin U \iff U \in U, \quad U \in U \Leftrightarrow U \notin U$$

Georg Cantor Definisce la tecnica di **diagonalizzazione**: data una lista infinita, è possibile creare un elemento che non appartiene a questa lista, utile per alcune dimostrazioni.

David Hilbert Vanno definite le regole in maniera chiara e univoca, per creare un formalismo di logica e deduzione veramente consistente e rigoroso. La matematica è una scienza che deve consentire un modo per dimostrare in maniera esatta i propri statement: devono quindi essere definiti, dalla comunità, assiomi e regole di deduzione di ciascun sistema. Vengono quindi definiti i problemi di Hilbert, una celebre lista di 23 quesiti matematici. Non tra questi, è il problema della decidibilità.

Kurt Gödel Dimostra il **teorema di incompletezza**, secondo cui non è possibile dimostrare la consistenza della matematica con i metodi dell'aritmetica (come David Hilbert sperava): alcune cose vanno date per buone! Meno cose possibili, ma qualcosa sì. Le dimostrazioni di consistenza dell'aritmetica utilizzano elementi di teoria degli ordinali¹. Il concetto di verità non può essere definito con metodi finiti. L'aritmetica non può dimostrare tutte le asserzioni: queste possono comunque essere vere!

¹La teoria degli ordinali, sviluppata da Cantor, generalizza il concetto di posizione (primo, secondo...) agli insiemi infiniti, andando oltre i numeri naturali. Un numero ordinale è un insieme ben ordinato, dove ogni ordinale è l'insieme di tutti gli ordinali precedenti

Alonzo Church Dimostra che l'aritmetica è indecidibile, introducendo un calcolo formale noto come λ -calcolo.

Alan Turing Un contemporaneo di Alonzo Church, formalizza la prima **macchina teorica universale**, nota come **Macchina di Turing**. Da lì nasce l'idea di creare un calcolatore capace di simulare tutte le altre macchine. Nasce l'**Halting Problem**, che si dimostra non risolvibile con il metodo di diagonalizzazione.

0.2 Cos'è un algoritmo?

Nonostante moltissimi algoritmi siano stati applicati negli anni per risolvere problemi tipici (le 4 operazioni, problema di Euclide, crivello di Eratostene, fattorizzazione o test di divisibilità), per molto tempo **non è stato definito il concetto di algoritmo**. Un algoritmo è una procedura descritta in maniera univoca usando un formalismo opportuno. Può anche essere linguaggio naturale, ma **nulla deve essere definito in maniera ambigua** (niente "sale q.b.").

0.2.1 Risolvere un algoritmo

Risolvere in positivo un algoritmo Significa dimostrare l'esistenza di un algoritmo. Questa coincide con l'algoritmo stesso. È sufficiente esibire l'algoritmo stesso in un formalismo coerente e univoco.

Risolvere in negativo un algoritmo Significa dimostrare che tra tutti gli algoritmi possibili, nessuno risolve il problema posto. La dimostrazione di non-esistenza di algoritmo deve essere necessariamente accompagnata da una definizione univoca di algoritmo tramite un opportuno **sistema di calcolo universale**.

0.2.2 Congettura di Church-Turing

Si dimostra che esiste un'equivalenza tra i vari sistemi formali definiti fino ad ora. La congettura di Church-Turing afferma che ogni sistema formale si rifà a quelli già noti. Si osserva che questa congettura può essere solo confutata, ma non dimostrata.

Capitolo 1

Funzioni computabili e URM

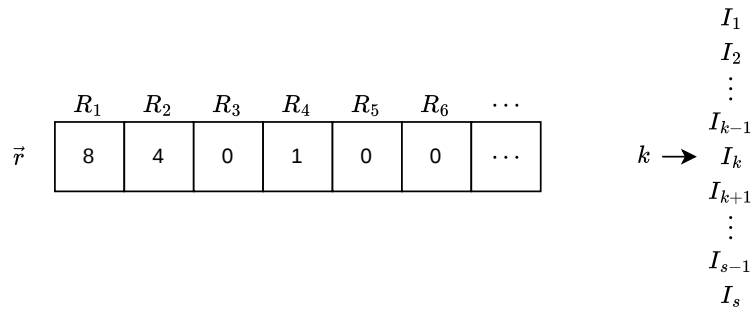
Apriamo il corso definendo l'idea di algoritmo, mostrando come questo possa essere descritto in maniera precisa usando un computer ideale (Macchina a Registri Illimitati), ponendo le basi per la teoria della computabilità.

1.1 Unlimited Register Machine

Una **Unlimited Register Machine** è una macchina teorica costituita da un array *potenzialmente* infinito, costituito da registri $R_1, R_2, \dots$ ciascuno dei quali può contenere un numero naturale a precisione infinita. L'informazione non è infinita, in quanto tutti i registri contengono il valore zero, a meno dei registri usati, i quali sono in un numero finito. Lo stato di una URM è contenuto nei suoi registri $\vec{r} \in \mathbb{N}^\infty$. Un algoritmo che cambia lo stato interno della URM è detto **URM-programma**. Un URM-programma è una sequenza finita di s istruzioni $I_1, I_2, \dots, I_s$.

1.1.1 Configurazione istantanea

È una coppia del tipo $(k, \vec{r}) \in \mathbb{N}^+ \times \mathbb{N}^\infty$, dove k si chiama **contatore di programma**, e rappresenta l'indice dell'istruzione I_k che sta per essere eseguita, mentre $\vec{r}$ è lo stato.



La configurazione istantanea corrente è $(k, (8, 4, 0, 1, 0, 0, \dots))$.

1.1.2 Istruzioni degli URM-programmi

Un *URM-programma* è una sequenza finita di s istruzioni $I_1, I_2, \dots, I_s$. Tra queste, abbiamo:

- Reset
- Assegnamento (non necessaria)
- Incremento
- Salto condizionato

Istruzione di Reset $Z(n)$. Azzerà il valore contenuto dello slot n -esimo.

$$(k, \vec{r}) \xrightarrow{Z(n)} (k+1, \vec{r}') \quad r'_i = \begin{cases} r_i, & i \neq n \\ 0, & i = n \end{cases}$$

Istruzione di Incremento $S(n)$. Incrementa di 1 il valore contenuto dello slot n -esimo.

$$(k, \vec{r}) \xrightarrow{S(n)} (k+1, \vec{r}') \quad r'_i = \begin{cases} r_i, & i \neq n \\ r_i + 1, & i = n \end{cases}$$

Istruzione di Assegnamento $T(m, n)$. Assegna allo slot n -esimo il valore contenuto nello slot m -esimo.

$$(k, \vec{r}) \xrightarrow{T(m, n)} (k+1, \vec{r}') \quad r'_i = \begin{cases} r_i, & i \neq n \\ r_m, & i = n \end{cases}$$

Istruzione di Salto condizionato $J(m, n, q)$. Lo stato rimane lo stesso, ma cambia il contatore di programma k .

$$(k, \vec{r}) \xrightarrow{J(m, n, q)} (k', \vec{r}) \quad k' = \begin{cases} k+1, & m \neq n \\ q, & m = n \end{cases}$$

1.2 Definizione di computazione

Sia $P = \{I_1, I_2, \dots, I_s\}$ un **programma** con s istruzioni, e $\vec{r}$ lo stato dei registri. La computazione $P(\vec{r})$ di P a partire dallo stato $\vec{r}$ è la sequenza (finita o infinita) di configurazioni istantanee con $k_1 = 1, \vec{r}^{(1)} = \vec{r}$.

$$(k_1, \vec{r}^{(1)}), (k_2, \vec{r}^{(2)}), (k_3, \vec{r}^{(3)}), \dots, (k_z, \vec{r}^{(z)}), (k_{z+1}, \vec{r}^{(z+1)}), \dots$$

tra queste configurazioni, identifichiamo

- Configurazione iniziale, $(1, \vec{r})$.
- Configurazione i -esima con successore, $(k_i, \vec{r}^{(i)})$. Se $k_i \leq s$, allora ha un successore nella computazione $P(\vec{r})$, e si ha $(k_i, \vec{r}^{(i)}) \xrightarrow{I_{k_i+1}} (k_{i+1}, \vec{r}^{(i+1)})$
- Configurazione finale, $(k_i, \vec{r}^{(i)})$. Se $k_i > s$, allora non ha un successore nella computazione $P(\vec{r})$, e quindi è la **configurazione finale** della stessa. Si arriva alla configurazione finale tramite esecuzione lineare o salti condizionali.

introduciamo quindi una notazione: $P(\vec{r})$ è un programma con stato iniziale $\vec{r}$,

$$r_i = \begin{cases} a_i & \text{se } i < n \\ 0 & \text{altrimenti} \end{cases}$$

$$P(a_1, a_2, \dots, a_n) \equiv P(a_1, a_2, \dots, a_n, 0, 0, \dots)$$

In particolare, possiamo indicare se la computazione termina o meno.

- $P(\vec{r}) \downarrow$ è una computazione **terminante**;
- $P(\vec{r}) \uparrow$ è una computazione **non terminante**

Solitamente, il registro R_1 viene usato per l'input e l'output della computazione. Nella notazione, scriveremo che la computazione che torna b , $P(\vec{r}) \downarrow b$.

1.3 Funzione parziale

Definiamo $f : A \rightarrow B$ funzione parziale, tale che $\text{Dom}(f) \subseteq A$, per cui il dominio potrebbe non essere tutto A . Per $a \in A$, se f è definita su a , $f(a) \downarrow$, altrimenti $f(a) \uparrow$. Con questa notazione che riprende la terminazione (o non terminazione) dei programmi URM, possiamo indicare se una funzione è definita per un determinato valore dell'insieme A .

Se $f, g : A \rightarrow B$ funzioni parziali, diremo che $f \simeq g$ se:

1. $\forall a \in A : f(a) \downarrow \Leftrightarrow g(a) \downarrow$
2. $\forall a \in A : f(a) \downarrow \implies f(a) = g(a)$

Questa viene chiamata **equivalenza di Kleene**, o solo equivalenza.

1.3.1 Funzioni URM-calcolabili

Sia P un programma. $\forall n \geq 1$, poniamo

$$f_P^{(n)}(a_1, a_2, \dots, a_n) = \begin{cases} b & \text{se } P(a_1, a_2, \dots, a_n) \downarrow b \\ \uparrow & \text{se } P(a_1, a_2, \dots, a_n) \uparrow \end{cases}$$

chiameremo la funzione parziale $f_P^{(n)} : \mathbb{N}^n \rightarrow \mathbb{N}$ la funzione n -aria calcolata dal programma P . Una funzione parziale g si dirà **URM-calcolabile** se esiste un programma URM P tale che $g \simeq f_P^{(n)}$.

L'insieme $\mathcal{C}^{\text{URM}}$ e $\mathcal{C}_n^{\text{URM}}$ La collezione di tutte le funzioni URM-calcolabili di qualunque arietà si indica con $\mathcal{C}^{\text{URM}}$. La collezione di tutte le funzioni URM calcolabili n -arie si indica con $\mathcal{C}_n^{\text{URM}}$. Scopriremo poi, con la congettura di Church-Turing, che $\mathcal{C}^{\text{URM}} = \mathcal{C}$.

$\mathcal{C}$ è enumerabile Dimostriamo ciò osservando che $\mathcal{C} = \bigcup_{n=1}^{\infty} \mathcal{C}_n^{\text{URM}}$, e che ciascuna delle $\mathcal{C}_n$ è costituita da istruzioni del tipo $S(n), Z(n), T(m, n), J(m, n, q)$, che sono enumerabili. Questo vuol dire che il dominio di $\mathcal{C}^{\text{URM}}$ è enumerabile, meno denso dell'insieme di tutte le funzioni f .

1.4 Esercizi

Si allega interprete di programmi-URM online col quale verificare le proprie soluzioni: <https://sites.oxy.edu/rnaimi/home/URMsim.htm>

1.4.1 Programmi URM

Si dimostri che le seguenti funzioni sono calcolabili esibendo opportuni programmi URM.

Funzione segno

$$f(x) = \begin{cases} 0 & \text{se } x = 0 \\ 1 & \text{se } x \neq 0 \end{cases}$$

Algoritmo 1 Funzione segno

- 1: $J(1, 2, 4)$
 - 2: $S(2)$
 - 3: $T(2, 1)$
-

Funzione costante

$$f(x) = 5$$

Algoritmo 2 Funzione costante

- 1: $Z(1)$
 - 2: $S(1)$
 - 3: $S(1)$
 - 4: $S(1)$
 - 5: $S(1)$
 - 6: $S(1)$
-

Funzione uguaglianza

$$f(x, y) = \begin{cases} 0 & \text{se } x = y \\ 1 & \text{se } x \neq y \end{cases}$$

Algoritmo 3 Funzione uguaglianza

1: $T(1, 3)$
2: $J(2, 3, 5)$
3: $Z(1)$
4: $S(1)$

Funzione minore o uguale

$$f(x, y) = \begin{cases} 0 & \text{se } x \leq y \\ 1 & \text{se } x > y \end{cases}$$

Algoritmo 4 Funzione minore o uguale

1: $T(1, 3)$
2: $Z(1)$
3: $S(2)$
4: $S(4)$
5: $J(2, 4, 8)$
6: $J(3, 4, 9)$
7: $J(1, 1, 4)$
8: $S(1)$

Funzione $x/3$

$$f(x) = \begin{cases} \frac{1}{3}x & \text{se } x \text{ è un multiplo di } 3 \\ \uparrow & \text{altrimenti} \end{cases}$$

Algoritmo 5 Funzione $x/3$

1: $T(1, 4)$
2: $Z(1)$
3: $S(1)$
4: $S(5)$
5: $S(5)$
6: $S(5)$
7: $J(4, 5, 9)$
8: $J(1, 1, 3)$

Funzione $\lfloor 2x/3 \rfloor$

$$f(x) = \begin{cases} \lfloor \frac{2}{3}x \rfloor & \text{se } x \text{ è un multiplo di } 3 \\ \uparrow & \text{altrimenti} \end{cases}$$

Algoritmo 6 Funzione $\lfloor 2x/3 \rfloor$

1: TODO

1.4.2 Dimostrazioni**L'istruzione T non è necessaria**

Algoritmo 7 Istruzione T

1: $Z(m)$
2: $S(m)$
3: $J(m, n, 1)$

Programma senza istruzioni J

Sia P un programma URM che non contiene alcuna istruzione del tipo $J(m, n, p)$, si dimostra che esiste $m \in \mathbb{N}$ tale che:

1.5 Correttezza dei programmi

Correttezza parziale Un programma P si dice **parzialmente corretto** rispetto a date specifiche di input α e di output β (note rispettivamente come **precondizione** e **postcondizione**) se, per ogni input $\vec{x}$ soddisfacente le specifiche α , se l'esecuzione di P su $\vec{x}$ è terminante, allora l'output di P soddisfa le specifiche β .

$$(\forall \vec{x})(\forall b)[(\alpha(\vec{x}) \wedge P(\vec{x}) \downarrow b) \rightarrow \beta(\vec{x}, b)]$$

Correttezza totale Un programma P si dice totalmente corretto rispetto alle specifiche α e β se è parzialmente corretto e vale anche la condizione di terminazione:

$$(\forall \vec{x})(\alpha(\vec{x}) \rightarrow P(\vec{x}) \downarrow)$$

1.5.1 Esempio di specifiche

Correttezza parziale su funzione sottrazione $\lambda xy.x - y$. Precondizione: $\alpha(x, y) \equiv x \geq 0 \wedge y \geq 0$. Postcondizione $\beta(x, y, z) \equiv z = x - y$.

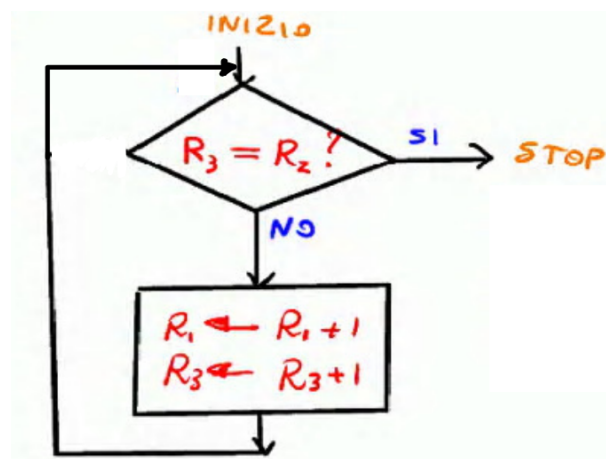
Correttezza totale su funzione sottrazione $\lambda xy.x - y$. Precondizione: $\alpha(x, y) \equiv x \geq y \wedge y \geq 0$. Postcondizione $\beta(x, y, z) \equiv z = x - y$

1.5.2 Verifica della correttezza parziale dei programmi di Floyd/Hoare

Introduciamo adesso delle metodologie per trovare un'asserzione per cui le condizioni di verifica del programma risultino corrette. Usiamo il seguente programma come esempio:

$$\lambda r_1 r_2.(r_1 + r_2)$$

- 1: J(3, 2, 5)
- 2: S(1)
- 3: S(3)
- 4: J(1, 1, 1)



$$\text{precondizione : } \begin{cases} r_1 = \bar{r}_1 \\ r_2 = \bar{r}_2 \\ r_3 = 0 \end{cases}$$

1.5.3 Verifica della correttezza totale dei programmi

Se la verifica della correttezza parziale sfrutta la logica, per la correttezza totale si vuole uno strumento che **verifica la terminazione** in ogni caso.

È sufficiente trovare una misura a valori in un insieme ben fondato che decresca strettamente ad ogni iterazione. Possiamo quindi rafforzare l'invariante A con la condizione di decrescita, ottenendo A^* .

1.5.4 Sulla verifica della correttezza

In generale, metteremo un'asserzione per ogni ciclo condizionale. Il numero di condizioni, invece, è uguale al numero di percorsi.

1.6 Predicati

Un predicato n -ario è una funzione $M : N \rightarrow \{true, false\}$. I predicati sono trattati come funzioni, associando ad essi la loro **funzione caratteristica**.

$$c_m(\vec{x}) := \begin{cases} 1 & \text{se } M(\vec{x}) = true \\ 0 & \text{se } M(\vec{x}) = false \end{cases}$$

La funzione caratteristica del predicato $x = 0$, ad esempio, è data da

$$g(x) = \begin{cases} 1 & \text{se } x = 0 \\ 0 & \text{altrimenti} \end{cases}$$

1.6.1 Predicati decidibili

Un predicato si dice **decidibile** se la sua funzione caratteristica è **calcolabile**. Scrivere un programma P che calcola la funzione caratteristica è sufficiente per dimostrare che il predicato sia decidibile.

1.7 Calcolabilità su altri domini

È possibile generalizzare i concetti di decidibilità e calcolabilità ai predicati $M : D \rightarrow \{true, false\}$ e funzioni $f : D \rightarrow D$ per domini $D \neq \mathbb{N}$: deve esistere tuttavia un modo effettivo per mappare il dominio di D su $\mathbb{N}$.

$$\mathbb{N} \xrightarrow{\alpha^{-1}} D \xrightarrow{f} D \xrightarrow{\alpha} \mathbb{N}$$

Si richiede una codifica effettiva di $D : \alpha : D \rightarrow \mathbb{N}$ che sia

- Iniettiva;
- Suriettiva;
- Effettivamente¹ calcolabile e invertibile.

Definiamo quindi una funzione $f^* = \alpha(f(\alpha^{-1}(x)))$. Questa funzione è calcolabile. Un esempio di codifica effettiva per lavorare in $\mathbb{Z}$ è

$$\alpha(x) = \begin{cases} 2x & \text{se } x \geq 0 \\ -2x - 1 & \text{altrimenti} \end{cases}, \quad \alpha^{-1}(y) = \begin{cases} \frac{1}{2}y & \text{se } y \text{ è pari} \\ -\frac{1}{2}(y+1) & \text{altrimenti} \end{cases}$$

¹Per effettivo, intendiamo che deve esistere un modo di calcolare la mappatura e l'inversione, un algoritmo vero e proprio.

Capitolo 2

Creare funzioni computabili

Tratteremo adesso una serie di metodi per combinare funzioni computabili, ottenendo altre funzioni computabili. Questo ci consentirà di dimostrare che moltissime funzioni sono computabili, senza dover scrivere un programma P ogni volta, rendendo meno tedioso il processo di dimostrazione della computabilità. Prima di definire gli strumenti usati, dobbiamo dimostrare che questi sono validi.

2.1 Condizioni di esistenza di programmi

Definiremo degli strumenti per capire se una funzione è calcolabile **senza dover scrivere il programma** che la calcola.

Insieme delle funzioni di base È un insieme di funzioni notoriamente calcolabili.

- Funzione identicamente nulla: $\underline{0}(x) = 0$
- Funzione successore: $\text{Incr}(x) = x + 1$
- Proiezioni: $\bigcup_i^n (x_1, \dots, x_n) = x_i$

Ciascuna di queste funzioni di base è calcolabile, e i programmi che la calcolano sono programmi a una sola istruzione ($Z(1), S(1), T(i, 1)$).

Osservazione interessante: la funzione identità è uguale a $\bigcup_1^1(x_1)$, ossia $T(1, 1)$.

2.1.1 Operazioni su cui le funzioni calcolabili sono chiuse

L'insieme delle funzioni calcolabili è chiuso rispetto alle seguenti operazioni.

Composizione Siano $f(y_1, \dots, y_k)$ funzione calcolabili con arietà k , $g_1(\vec{x}), \dots, g_k(\vec{x})$ k funzioni calcolabili con arietà n ($\vec{x} = (x_1, \dots, x_n)$). Allora, la funzione

$$h(\vec{x}) =_{def} f(g_1(\vec{x}), \dots, g_k(\vec{x}))$$

è **ottenuta per composizione** da $f, g_1, \dots, g_k$. $\lambda \vec{x}. f(g_1(\vec{x}), \dots, g_k(\vec{x}))$, ed è **calcolabile**. Si osservi che la funzione $h(\vec{x})$ è definita (ossia $h(\vec{x}) \downarrow$) se e solo se

- $g_1(\vec{x}) \downarrow, \dots, g_k(\vec{x}) \downarrow$
- $f(g_1(\vec{x}) \downarrow, \dots, g_k(\vec{x}) \downarrow)$

Ricorsione primitiva Permette di costruire funzioni espresse in termini di altre funzioni. Siano $f(\vec{x}), g(\vec{x}, y, z)$ funzioni assegnate, con f funzione n -aria ($n \geq 0$). Allora:

- $h(\vec{x}, 0) = f(\vec{x})$
- $h(\vec{x}, y + 1) = g(\vec{x}, y, h(\vec{x}, y))$

scopriremo che la ricorsione primitiva non esaurisce tutte le funzioni ricorsive.

Minimalizzazione Data una funzione $f(\vec{x}, y)$ (anche parziale), poniamo

$$\mu y(f(\vec{x}, y) = 0) = \begin{cases} \text{minimo } y \text{ tale che} \\ \cdot f(\vec{x}, z) \downarrow \text{ per ogni } z < y \\ \cdot f(\vec{x}, y) = 0, \quad \text{se un siffatto } y \text{ esiste} \\ \uparrow \quad \text{altrimenti} \end{cases}$$

dove μ è detto operatore di minimalizzazione. A parole, ritorna il valore più piccolo di y per cui, tutti i valori z minori o uguali di y terminano la funzione¹ f , e tale che $f(\vec{x}, y) = 0$. Altrimenti non ritorna.

¹Questa condizione, a mia opinione, un po' strana, si rifà al fatto che il programma che calcola la minimalizzazione di una funzione f inizia a calcolarla dal basso verso l'alto. Se la funzione non termina per una qualsiasi $x < y$ che stiamo cercando, allora la minimalizzazione non termina.

Capitolo 3

Composizione di programmi

La prima tecnica che andremo ad approfondire, sarà la **composizione di programmi**. Per arrivare al concetto di **composizione**, dovremmo tuttavia introdurre alcuni concetti.

3.1 Come concatenare programmi

Siano P e Q due programmi (con P di lunghezza s). Desideriamo concatenare P e Q in modo che Q venga eseguito correttamente non appena P termina. Ci sono delle osservazioni fondamentali da fare:

1. Il programma P deve fermarsi sempre al contatore di programma $s + 1$.
2. Le istruzioni di salto di P devono essere circonscritte a P . Lo stesso vale per Q . Bisognerà modificare in maniera opportuna le etichette di salto q delle istruzioni J nel programma Q .

Risolveremo ciascuno di questi problemi.

Programmi in forma standard - Soluzione al problema 1 Un programma P si dice in forma standard se il contatore di programma si ferma sempre a $s + 1$. È inoltre dimostrabile che, dato un programma $P = I_1, I_2, \dots, I_s$, esiste sempre un programma $P^* = I_1^*, I_2^*, \dots, I_s^*$ equivalente¹ in forma standard, tale che

$$I_j^* = \begin{cases} J(m, n, s + 1) & \text{se } I_j = J(m, n, q) \text{ con } q > s + 1 \\ I_j & \text{altrimenti} \end{cases}$$

ed è dimostrabile per induzione. Abbiamo quindi sempre la possibilità di rendere un programma non-in forma standard, in un programma in forma standard.

Correzione dei salti - Soluzione al problema 2 Dato un programma P indichiamo con $(P + l)$ il programma che si ottiene sostituendo in P ogni istruzione $J(m, n, q)$ con $J(m, n, q + l)$, lasciando invariate le altre istruzioni.

3.1.1 Concatenazione di programmi

Dati due programmi P e Q , con $s = \text{length}(P)$, la **concatenazione** di P e Q , scritta PQ , è il programma le cui prime s istruzioni coincidono con quelle di P e le cui successive istruzioni coincidono con quelle di $(Q + S)$. Due proprietà interessanti, sono:

- $(PQ)R = P(QR)$, ossia la concatenazione è associativa;
- Se P e Q sono in forma standard, anche PQ è in forma standard.

3.1.2 Problema della concatenazione

La concatenazione di due programmi non equivale alla composizione delle rispettive funzioni: le aree di memoria su cui operano potrebbero sovrapporsi, portando a risultati inaspettati o perdita di dati. Andremo quindi a stabilire un modo per delimitare l'area di memoria di ciascuna delle funzioni.

¹ogni computazione è identica, a meno del contatore di programma della configurazione finale

Programma che isola i programmi Utilizziamo la seguente notazione $R_l \leftarrow f_P^{(n)}(R_{l_1}, \dots, R_{l_n})$ per indicare il programma che

1. Copia il contenuto dai registri $R_{l_1}, \dots, R_{l_n}$ in $R_1, \dots, R_n$.
2. Cancella i registri di lavoro necessari per P , quelli che vanno da $(n+1)$ a $\rho(P)$. Partiamo da $(n+1)$ per non sovrascrivere quanto appena copiato.
3. Computa la funzione calcolata dal programma P .
4. Copia l'output di P nel registro R_l .

Tramite questa macro, possiamo finalmente descrivere con facilità la composizione di funzioni.

3.2 Composizione di funzioni (o sostituzione)

Siano $f(y_1, \dots, y_k)$ funzione calcolabile con arietà k , $g_1(\vec{x}), \dots, g_k(\vec{x})$ k funzioni calcolabili con arietà n ($\vec{x} = (x_1, \dots, x_n)$). Allora, la funzione

$$h(\vec{x}) =_{def} f(g_1(\vec{x}), \dots, g_k(\vec{x}))$$

è **ottenuta per composizione** da $f, g_1, \dots, g_k$. $\lambda \vec{x}. f(g_1(\vec{x}), \dots, g_k(\vec{x}))$, ed è **calcolabile**. Si osservi che la funzione $h(\vec{x})$ è definita (ossia $h(\vec{x}) \downarrow$) se e solo se

- $g_1(\vec{x}) \downarrow, \dots, g_k(\vec{x}) \downarrow$
- $f(g_1(\vec{x}) \downarrow, \dots, g_k(\vec{x}) \downarrow) \downarrow$

Abbiamo gli strumenti opportuni per dimostrare che l'insieme delle funzioni URM-calcolabili è chiuso rispetto alla composizione di funzioni URM-calcolabili.

Dimostrazione Siano $F, G_1, \dots, G_k$ programmi **in forma standard** che calcolano rispettivamente le funzioni $f, g_1, \dots, g_k$. Sia $m = \max(\rho(G_1), \dots, \rho(G_k), k, n)$, con ρ funzione che calcola il più grande indice utilizzato durante la computazione del programma passato come argomento. Oltre questo indice, la memoria è inutilizzata.

Allora, il programma

Algoritmo 8 *Composizione di funzioni*

$T(1, m+1)$	
$\vdots$	$\triangleright$ Copia l'input in un'area sicura
$T(n, m+n)$	
$R_{m+n+1} \leftarrow f_{G_1}(R_{m+1}, \dots, R_{m+n})$	
$\vdots$	$\triangleright$ Calcola le singole funzioni $g_1, \dots, g_k$
$R_{m+n+k} \leftarrow f_{G_k}(R_{m+1}, \dots, R_{m+n})$	
$R_1 \leftarrow f_F(R_{m+n+1}, \dots, R_{m+n+k})$	$\triangleright$ Calcola f sui valori di $g_1, \dots, g_k$ calcolati

calcola la funzione h , come volevasi dimostrare.

3.3 Sostituzioni di variabili

Le seguenti sostituzioni di variabili

- $h_1(x_1, x_2) =_{def} f(x_2, x_1)$ (permutazione)
- $h_2(x) =_{def} f(x, x)$ (identificazione)
- $h_3(x_1, x_2, x_3) =_{def} f(x_2, x_3)$ (introduzione di variabili "dummy", mute)

generano funzioni calcolabili, e sono casi particolari del seguente corollario.

Corollario

Sia $f(y_1, \dots, y_k)$ una funzione calcolabile, e siano $x_{i_1}, x_{i_2}, \dots, x_{i_k}$ una sequenza di variabili con $i_1, \dots, i_k \in \{1, \dots, n\}$ (con possibili ripetizioni). Sia $h_x(x_1, \dots, x_n) =_{def} f(x_{i_1}, \dots, x_{i_k})$. Allora h è calcolabile.

Dimostrazione Basta osservare che $h(\vec{x}) = f(\bigcup_{i_1}^n, \dots, \bigcup_{i_k}^n)$, con $\vec{x} = (x_1, \dots, x_n)$, è calcolabile.

Capitolo 4

Ricorsione

La ricorsione permette di definire nuovi valori di una funzione mediante vecchi valori. Tra queste funzioni, abbiamo la funzione fattoriale o la funzione per i numeri di Fibonacci.

4.1 Ricorsione primitiva

La definizione per ricorsione che useremo è la **ricorsione primitiva**. Siano $f(\vec{x})$ e $g(\vec{x}, y, z)$ funzioni. Allora

$$\begin{cases} h(\vec{x}, 0) = f(\vec{x}) \\ h(\vec{x}, y + 1) = g(\vec{x}, y, h(\vec{x}, y)) \end{cases}$$

Costituisce una definizione per ricorsione di $h(\vec{x}, y)$ da $f(\vec{x}), g(\vec{x}, y, z)$. Le due equazioni sono dette **equazioni ricorsive**.

4.1.1 Teorema sull'esistenza e unicità

La precedente definizione trae forza dal seguente teorema. Sia $\vec{x} = (x_1, \dots, x_n)$, e $f(\vec{x})$ e $g(\vec{x}, y, z)$ funzioni. Allora esiste un'unica funzione $h(\vec{x}, y)$ che soddisfa le equazioni ricorsive

- $h(\vec{x}, 0) = f(\vec{x})$
- $h(\vec{x}, y + 1) = g(\vec{x}, y, h(\vec{x}, y))$

Inoltre, nel caso specifico di $\vec{x}$ assenti, e quindi arietà nulla, le equazioni ricorsive diventano

$$\begin{cases} h(0) = a \\ h(y + 1) = g(y, h(y)) \end{cases}$$

Unicità Siano h' e h'' soddisfacenti le proprietà

- $h(\vec{x}, 0) = f(\vec{x})$
- $h(\vec{x}, y + 1) = g(\vec{x}, y, h(\vec{x}, y))$

e tali che $h' \neq h''$. Sia y il **minimo** per cui esista $\vec{x}$ tale che

$$h'(\vec{x}, y) \neq h''(\vec{x}, y)$$

Questo valore di y deve essere maggiore (e non uguale) a 0, in quanto questo statement

$$f(\vec{x}) = h'(\vec{x}, 0) \neq h''(\vec{x}, 0) = f(\vec{x})$$

è ovviamente un assurdo. Sia dunque $y = y^* + 1$, con $y^* \geq 0$. Pertanto, $h'(\vec{x}, y^*) = h''(\vec{x}, y^*)$, da cui

$$\begin{aligned} h'(\vec{x}, y) &= h(\vec{x}, y^* + 1) \\ &= g(\vec{x}, y^*, h'(\vec{x}, y^*)) \\ &= g(\vec{x}, y^*, h''(\vec{x}, y^*)) \\ &= h''(\vec{x}, y^*). \quad \textbf{Contraddizione!} \end{aligned}$$

$h' = h''$. Quindi, se esiste una funzione h che soddisfa delle date equazioni ricorsive, questa è unica.

Esistenza La dimostrazione dell'esistenza è data per buona, in quanto molto complessa da affrontare.

4.1.2 Esempi di funzioni calcolabili con ricorsione primitive

Addizione, moltiplicazione, fattoriale. TODO.

4.1.3 Teorema sulla calcolabilità della ricorsione primitiva

Se $f(\vec{x})$ e $g(\vec{x}, y, z)$ sono calcolabili, anche la funzione $h(\vec{x}, y)$ definita per ricorsione primitiva con $f(\vec{x})$ e $g(\vec{x}, y, z)$ come equazioni ricorsive, è calcolabile.

Dimostrazione Siano F e G programmi in **forma standard** che calcolano rispettivamente f e g . Poniamo $m = \max(\rho(F), \rho(G), n+2)$, ove $\vec{x} = (x_1, \dots, x_n)$. È facile convincersi che il seguente programma calcola la funzione $h(\vec{x}, y)$.

Algoritmo 9 Ricorsione primitiva

$T(1, m+1)$	
$\vdots$	
$T(n+1, m+n+1)$	▷ Copia l'input in un'area sicura
$R_{m+n+3} \leftarrow f_F^{(n)}(R_{m+1}, \dots, R_{m+n})$	▷ Calcola il caso base
$[I_q]:$	
$J(m+n+2, m+n+1, p)$	▷ Condizione di uscita dalla ricorsione
$R_{m+n+3} \leftarrow f_G^{(n+2)}(R_{m+1}, \dots, R_{m+n}, R_{m+n+2}, R_{m+n+3})$	
$S(m+n+2)$	
$J(1, 1, q)$	▷ while True
$[I_p]: T(m+n+3, 1)$	

Il registro R_{m+n+1} contiene il numero di passi ricorsivi da fare, con R_{m+n+2} usato come contatore.

4.2 Funzioni primitive ricorsive

Abbiamo dimostrato, fino ad ora, che le funzioni iniziali sono calcolabili, le composizioni di funzioni calcolabili sono calcolabili, e che **la ricorsione primitiva** su funzioni calcolabili dà luogo a funzioni **calcolabili**. Chiameremo **primitive ricorsive** le funzioni ottenute a partire da funzioni iniziali tramite un numero finito di composizioni e ricorsioni primitive. Denotiamo l'insieme delle primitive ricorsive $\mathcal{PR}$.

4.2.1 Insieme $\mathcal{PR}$

Non tutte le funzioni URM-calcolabili appartengono all'insieme delle funzioni primitive ricorsive. La minimalizzazione (non limitata) ci permetterà di ottenere funzioni molto più espressive, permettendoci di solcare anche il suolo delle funzioni parziali **non totali**.

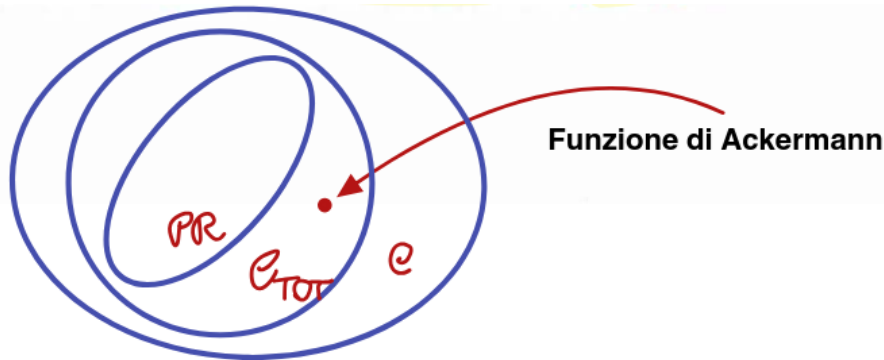


Immagine 4.1: Le funzioni $\mathcal{PR}$ non coprono tutte le funzioni calcolabili totali, e in particolare, nessuna funzione parziale **non totale** è primitiva ricorsiva.

Capitolo 5

Alcuni predicati decidibili e funzioni calcolabili

5.1 Funzioni definite per casi

Siano $f_1(\vec{x}), \dots, f_k(\vec{x})$ funzioni **calcolabili totali** e $M_1(\vec{x}), \dots, M_k(\vec{x})$ sono predicati **decidibili** tali che, per ogni $\vec{x}$, **esattamente uno** tra $M_1(\vec{x}), \dots, M_k(\vec{x})$ è vero. Allora, la funzione

$$g(\vec{x}) = \begin{cases} f_1(\vec{x}) & \text{se } M_1(\vec{x}) \text{ è vero} \\ \vdots & \vdots \\ f_k(\vec{x}) & \text{se } M_k(\vec{x}) \text{ è vero} \end{cases}$$

è calcolabile.

Dimostrazione Basta osservare che, con $c_{M_i}(\vec{x})$ funzione caratteristica del predicato $M_i(\vec{x})$, la seguente funzione è calcolabile

$$g(\vec{x}) = c_{M_1}(\vec{x}) \cdot f_1(\vec{x}) + \dots + c_{M_k}(\vec{x}) \cdot f_k(\vec{x})$$

La calcolabilità della funzione $g(\vec{x})$ è verificata in quanto:

- Ogni termine $c_{M_i}(\vec{x}) \cdot f_i(\vec{x})$ è ottenuto dalla composizione di tre funzioni, ossia: la funzione caratteristica del predicato c_{M_i} (calcolabile in quanto decidibile per ipotesi), la funzione $f_i(\vec{x})$ (calcolabile per ipotesi) e la **funzione prodotto** (calcolabile).
- La **somma di funzioni** calcolabili è calcolabile, in quanto ottenibile per composizione.

5.2 Algebra della decidibilità

Se $M(\vec{x})$ e $Q(\vec{x})$ sono predicati **decidibili**, allora anche i seguenti predicati sono decidibili

- $\neg M(\vec{x})$: $C_{\neg M}(\vec{x}) = 1 \div M(\vec{x})$
- $M(\vec{x}) \wedge Q(\vec{x})$: $M(\vec{x}) \cdot Q(\vec{x})$
- $M(\vec{x}) \vee Q(\vec{x})$: $\text{sign}(M(\vec{x}) + Q(\vec{x}))$, oppure $\max(M(\vec{x}), Q(\vec{x}))$

5.3 Sommatorie e produttorie limitate

Tramite ricorsione, possiamo definire le operazioni di sommatoria e produttoria limitate, ossia:

$$\sum_{z < y} f(\vec{x}, z), \quad \prod_{z < y} f(\vec{x}, z)$$

se $f(\vec{x}, z)$ è calcolabile, anche le rispettive sommatorie e produttorie limitate sono calcolabili.

Dimostrazione Basta osservare che le sommatorie e le produttorie limitate soddisfano queste ricorsioni primitive:

$$\begin{cases} \sum_{z < 0} f(\vec{x}, z) = 0 \\ \sum_{z < y+1} f(\vec{x}, z) = \sum_{z < y} f(\vec{x}, z) + f(\vec{x}, y) \end{cases} \quad \begin{cases} \prod_{z < 0} f(\vec{x}, z) = 1 \\ \prod_{z < y+1} f(\vec{x}, z) = \prod_{z < y} f(\vec{x}, z) \cdot f(\vec{x}, y) \end{cases}$$

sono quindi calcolabili. Lo stesso vale anche per

5.3.1 Corollario

Quanto detto in precedenza rispetto alle sommatorie e alle produttorie limitate, vale anche quando y viene sostituito con una funzione calcolabile $k(\vec{x}, \vec{w})$.

$$\sum_{z < k(\vec{x}, \vec{w})} f(\vec{x}, z), \quad \prod_{z < k(\vec{x}, \vec{w})} f(\vec{x}, z)$$

La dimostrazione è per composizione. Un caso particolare, è quello di

$$\sum_{z \leq y} f(\vec{x}, z) = \sum_{z < y+1} f(\vec{x}, z), \quad \prod_{z \leq y} f(\vec{x}, z) = \prod_{z < y+1} f(\vec{x}, z)$$

5.4 Minimalizzazione limitata

È una versione *bounded* dell'operazione di minimalizzazione, di cui parleremo nei prossimi capitoli. Indicato con $\mu z < y (f(\vec{x}, z) = 0)$, indica il minor z minore di y tale $f(\vec{x}, z) = 0$ ¹. La funzione in questione deve essere una **funzione totale** e **calcolabile**. In generale, diremo che

$$\mu z < y (f(\vec{x}, z) = 0) = \begin{cases} \text{minimo } z < y \text{ tale che } f(\vec{x}, z) = 0 & \text{se tale } z \text{ esiste.} \\ y & \text{altrimenti} \end{cases}$$

Dimostrazione Dobbiamo dimostrare che $\mu z < y (f(\vec{x}, z) = 0)$ sia calcolabile e totale. Che sia totale è triviale. Andiamo a costruire l'operatore di minimalizzazione limitata usando altre operazioni che sono notoriamente calcolabili.

$$\prod_{u < y} \text{sign}(f(\vec{x}, u)) = \begin{cases} 1 & \text{se } f(\vec{x}, u) \neq 0 \text{ per ogni } u < y \\ 0 & \text{altrimenti} \end{cases}$$

$$\sum_{z < y} \prod_{u < z} \text{sign}(f(\vec{x}, u)) = \begin{cases} y & \text{se } f(\vec{x}, u) \neq 0 \text{ per ogni } u < y \\ z_0 & \text{altrimenti, dove } z_0 \text{ è il minimo } z < y \text{ tale che } f(\vec{x}, z) = 0 \end{cases}$$

$f(\vec{x}, z)$

z	0	1	2	3	4	5	6	7	8	...
	4	2	1	7	0	6	5	0	2	...

$\prod_{u \leq z} \text{sg}(f(\vec{x}, u))$

z	0	1	2	3	4	5	6	7	8	...
	1	1	1	1	0	0	0	0	0	...

$\sum_{u < z} \prod_{u \leq v} \text{sg}(f(\vec{x}, u))$

z	0	1	2	3	4	5	6	7	8	...
	0	1	2	3	4	4	4	4	4	...

$\mu z < y (f(\vec{x}, z) = 0)$

z	0	1	2	3	4	5	6	7	8	...
	0	1	2	3	4	4	4	4	4	...

$\sum_{z < y} \prod_{u < z} \text{sign}(f(\vec{x}, u))$ è l'operatore di minimalizzazione limitata.

¹La condizione potrebbe essere diversa, ma questo è uno dei casi più tipici.

5.4.1 Corollario - minimalizzazione limitata da una funzione

Se $f(\vec{x}, z)$ e $k(\vec{x}, \vec{w})$ sono calcolabili, $\mu z < k(\vec{x}, \vec{w})(f(\vec{x}, z) = 0)$ è calcolabile.

5.4.2 Corollario - minimalizzazione di predicati decidibili

Se $M(\vec{x}, y)$ è calcolabile, $\mu z < y M(\vec{x}, y)$ è calcolabile, e coincide col minimo valore di z per cui il predicato è vero.

$$\mu z < y M(\vec{x}, y) = \mu z < y (C_M(\vec{x}, z) = 1) = \mu z < y (\overline{\text{sg}}(C_M(\vec{x}, z)) = 0).$$

5.5 Quantificatori limitati

Sia $R(\vec{x}, y)$ un predicato decidibile. Allora, i seguenti predicati sono decidibili:

$$(\forall z < y) R(\vec{x}, z), \quad (\exists z < y) R(\vec{x}, z)$$

Dimostrazione: per ogni Il predicato coincide con una produttoria limitata delle funzioni caratteristiche del predicato. È quindi calcolabile.

Dimostrazione: esiste un Il predicato coincide col segno di una sommatoria limitata delle funzioni caratteristiche del predicato. È quindi calcolabile.

5.6 Codifiche per tuple

In generale, sappiamo che è possibile codificare in maniera effettiva sequenze di numeri in un singolo numero (in maniera totalmente univoca). Questo vuol dire che funzioni che operano su tuple, sono calcolabili. Questo apre le porte anche all'adattamento di funzioni ricorsive non primitive, in funzioni ricorsive primitive (vedremo un esempio sulla funzione di Fibonacci).

5.6.1 Codifiche per coppie di numeri

Dimostriamo che la codifica $\pi(x, y) = 2^x(2y + 1) \div 1$ è una biiezione calcolabile.

Calcolabilità È banale, nasce infatti da combinazione di funzioni calcolabili.

Iniettività

$$\begin{aligned} \pi(x, y) &= \pi(x', y') \\ 2^x(2y + 1) \div 1 &= 2^{x'}(2y' + 1) \div 1 \\ 2^x(2y + 1) &= 2^{x'}(2y' + 1) \end{aligned}$$

DA FINIRE

5.6.2 Codifiche per sequenze di numeri

Vogliamo

Ogni numero $x \in \mathbb{N}$ può essere scritto nella forma

$$x = \sum_{i=0}^{\ell} \alpha_i 2^i, \quad \text{con } \alpha_i \in \{0, 1\}$$

univocamente.

Quindi ogni $x > 0$ ammette espressioni univoche della forma:

$$x = 2^{b_1} + 2^{b_2} + \dots + 2^{b_\ell}, \quad 0 \leq b_1 < b_2 < \dots < b_\ell, \ell \geq 1$$

$$x = 2^{a_1} + 2^{a_1+a_2+1} + \dots + 2^{a_1+\dots+a_\ell+(\ell-1)}, \quad a_i \geq 0, \ell \geq 1$$

Risultano quindi definite le seguenti funzioni:

$$\alpha(i, x) = \alpha_i \quad \left(\text{con } x = \sum_{i=0}^{\ell} \alpha_i 2^i \right)$$

$$\ell(x) = \begin{cases} \ell & \text{se } x > 0 \\ 0 & \text{altrimenti} \end{cases}$$

$$b(i, x) = \begin{cases} b_i & \text{se } x > 0 \text{ e } 1 \leq i \leq \ell(x) \\ 0 & \text{altrimenti} \end{cases}$$

$$a(i, x) = \begin{cases} a_i & \text{se } x > 0 \text{ e } 1 \leq i \leq \ell(x) \\ 0 & \text{altrimenti} \end{cases}$$

Capitolo 6

Minimalizzazione

Noto anche come **operatore di ricerca**.

6.1 Operazione di minimalizzazione

Mediante programmi URM è possibile effettuare la ricerca del più piccolo indice tale che si verifica una certa condizione. Non trovare l'indice porta a non terminazione. La **minimalizzazione** (non limitata) permette di definire una funzione $g(\vec{x})$ a partire da una funzione $f(\vec{x}, y)$ ponendo

$$g(\vec{x}) =_{def} \text{minimo } y \text{ tale che } f(\vec{x}, y) = 0$$

che in pseudo-codice equivale a:

Algoritmo 10 Minimalizzazione in pseudocodice

```
y = 0
while f(x), y ≠ 0 do
  y = y + 1
end while
return y
```

Definizione Data una funzione $f(\vec{x}, y)$ (anche parziale), poniamo

$$\mu y(f(\vec{x}, y) = 0) = \begin{cases} \text{minimo } y \text{ tale che} \\ \cdot f(\vec{x}, z) \downarrow \text{ per ogni } z < y \\ \cdot f(\vec{x}, y) = 0, \quad \text{se un siffatto } y \text{ esiste} \\ \uparrow \text{ altrimenti} \end{cases}$$

dove μ è detto operatore di minimalizzazione. L'insieme $\mathcal{C}_{URM}$ è chiuso rispetto alla minimalizzazione.

6.1.1 Dimostrazione della calcolabilità

Vogliamo calcolare il seguente programma: $g(\vec{x}) = \mu y(f(\vec{x}, z) = 0)$.

Algoritmo 11 Minimalizzazione

```
T(1, m + 1)
⋮
T(n, m + n)
[Ip]:
R1 ← fF(Rm+1, ..., Rm+n, Rm+n+1)
J(1, m + n + 2, q)
S(m + n + 1)
J(1, 1, p)
[Iq]: T(m + n + 1, 1)
```

6.1.2 Corollario sulla minimalizzazione dei predicati decidibili

Il minimo valore y tale che il predicato $R(\vec{x}, y)$ è vero, è calcolabile, in quanto

$$\mu y(\overline{\text{sign}}(C_R(\vec{x}, y)) = 0)$$

è calcolabile.

6.1.3 Funzioni parziali da funzioni totali

L'operatore di minimalizzazione, a causa della sua definizione, può derivare funzioni non totali a partire da funzioni totali!

Esempio Se infatti la funzione $f(x, y) = |x - y^2|$ risulta essere calcolabile e totale, la funzione $g(x, y) \simeq \mu y(f(x, y) = 0)$ è calcolabile, ma non totale! Quando x non è un quadrato perfetto, $\nexists y : y^2 = x \Rightarrow \nexists y = \sqrt{x}$. Quindi, la funzione calcolata a partire dalla f totale e calcolabile, è

$$g(x, y) = \begin{cases} \sqrt{x} & \text{se } x \text{ è un quadrato perfetto} \\ \uparrow & \text{altrimenti} \end{cases}$$

6.2 Funzioni ricorsive parziali

La famiglia delle **funzioni ricorsive parziali** è quella delle funzioni generate per

- Composizione
- Ricorsione primitiva
- Minimalizzazione non limitata

di **funzioni iniziali**. Scopriremo che questa famiglia coincide con la famiglia delle funzioni **URM-calcolabili**.

Capitolo 7

Completezza delle funzioni ricorsive parziali

Faremo vedere che la famiglia di funzioni ricorsive parziali, è uguale alla funzioni URM-calcolabili.

7.1 Funzioni ricorsive parziali

La famiglia delle funzioni ricorsive parziali è quella delle funzioni definibili a partire dalle funzioni iniziali e con

- Composizione
- Ricorsione primitiva
- Minimalizzazione (non limitata)

Il nostro obiettivo sarà dimostrare che famiglia di funzioni ricorsive parziali, è uguale alla funzioni URM-calcolabili. Con la tesi di Church, vedremo come parleremo di funzioni calcolabili, e non solo URM-calcolabili.

7.2 Funzione di Ackermann

La funzione di Ackermann permette di esplicitare il limite delle funzioni ricorsive primitive. Il motivo per cui non può essere espressa tramite ricorsione primitiva, è la doppia ricorsione. Anche la funzione dei numeri di Fibonacci può essere ricondotta ad una forma primitiva ricorsiva, ma questa funzione no.

$$\psi(x, y) = \begin{cases} \psi(0, y) = \begin{cases} 1 & \text{se } y = 0 \\ 2 & \text{se } y = 1 \\ y + 2 & \text{se } y > 1 \end{cases} & \text{se } x = 0 \\ \psi(x, 0) = 1 & \text{se } x \geq 1, y = 0 \\ \psi(x + 1, y + 1) = \psi(x, \psi(x + 1, y)) & \text{se } x \geq 1, y \geq 1 \end{cases}$$

per facilitare, cambiamo notazione. $\psi_x(y) \equiv \psi(x, y)$.

$$\psi_x(y) = \begin{cases} \psi_0(y) = \begin{cases} 1 & \text{se } y = 0 \\ 2 & \text{se } y = 1 \\ y + 2 & \text{se } y > 1 \end{cases} & \text{se } x = 0 \\ \psi_x(0) = 1 & \text{se } x \geq 1, y = 0 \\ \psi_{x+1}(y + 1) = \psi_x(\psi_{x+1}(y)) & \text{se } x \geq 1, y \geq 1 \end{cases}$$

osserviamo che $\psi_{x+1}(y + 1) = \psi_x(\psi_{x+1}(y)) = \psi_x^{(y+1)}(1)$. In generale dimostrare la calcolabilità di questa funzione non è facile.

7.2.1 Dimostrazione calcolabilità di $\lambda y.\psi_x(y)$

Faremo vedere per induzione su x che la restrizione (unaria) $\lambda y.\psi_x(y)$ è calcolabile e totale. Osserviamo che non stiamo andando a valutare la funzione al variare di x , ma stiamo valutando su valori fissati di x .

- Caso base: $\lambda y.\psi_0(y)$. Questo termine è già presente nella definizione per casi.

$$\psi_0(y) = \begin{cases} 1 & \text{se } y = 0 \\ 2 & \text{se } y = 1 \\ y + 2 & \text{se } y > 1 \end{cases}$$

- Passo induttivo: l'ipotesi induttiva è che $\psi_x(y)$ sia calcolabile e totale. Si osservi quindi che la funzione ψ_{x+1} può essere definita sulla seguente ricorsione primitiva (su y), in termini di ψ_x :

$$\begin{cases} \psi_{x+1}(0) = 1 \\ \psi_{x+1}(y + 1) = \psi_x(\psi_{x+1}(y)) \end{cases}$$

pertanto, per induzione, ψ_{x+1} è calcolabile e totale. Il fatto che questa funzione sia calcolabile per x fissati, non significa che la funzione di Ackermann sia calcolabile.

Addizione	Prodotto	Potenza	Tetrazione	Pentazione

7.2.2 Calcolabilità della funzione di Ackermann

Questa è la parte più difficile della dimostrazione.

Triple Intendiamo caratterizzare insiemi finiti S di triple $(x, y, z) \in \mathbb{N}$ tali che, $\forall (x, y, z) \in S$:

- $z = \psi(x, y)$
- S contiene **tutte** le triple $(\bar{x}, \bar{y}, \psi(\bar{x}, \bar{y}))$ necessarie al calcolo di $\psi(x, y)$.

questi insiemi devono inoltre essere **validi**.

Insiemi validi Un insieme di triple S è detto valido se le seguenti condizioni sono soddisfatte:

- se $(0, y, z) \in S$, allora $z = \begin{cases} 1 & \text{se } y = 0 \\ 2 & \text{se } y = 1 \\ y + 2 & \text{se } y > 1 \end{cases}$.
- se $(x, 0, z) \in S$, allora $z = 1$.
- se $(x + 1, y + 1, z) \in S$, allora $\exists u \in \mathbb{N} : (x + 1, y, u), (x, u, z)$.

Osserviamo che $\psi(x + 1, y + 1) = \psi(\underbrace{x, \psi(x + 1, y)}_w)$. Un insieme finito di triple S è detto **valido**

rispetto alla coppia $(x, y) \in \mathbb{N}^2$ se

- S è valido.
- $\exists u \in \mathbb{N} : (x, y, u) \in S$.

Possiamo poi definire un buon ordinamento $<_{\text{lex}}$ su $\mathbb{N} \times \mathbb{N}$.

$$(x, y) <_{\text{lex}} (x', y') \iff (x < x') \vee (x = x' \wedge y < y').$$

Fatto 1 Se S è valido, allora $\forall(x, y, z) \in S \Rightarrow z = \psi(x, y)$. Si dimostra per assurdo.

TODO

Fatto 2 $\forall(x, y) \in \mathbb{N}^2, \exists S$ finito valido per (x, y) . Si dimostra per assurdo.

TODO

Passo finale della dimostrazione Una tripla (x, y, z) può essere codificata in un singolo numero positivo $u = 2^x 3^y 5^z$, mentre un insieme finito di numeri positivi, e quindi di triple, può essere codificato in un singolo numero $v = p_{u_1} p_{u_2} \dots p_{u_k}$. Il set di triple codificato da v è indicato con S_v .

Quindi, $(x, y, z) \in S_v \Leftrightarrow p_{2^x 3^y 5^z}$ divide v . Essendo questa una mutua implicazione, la decidibilità di un predicato implica la decidibilità dell'altro. Definiamo quindi la funzione

$$R(x, y, v) \equiv "v \text{ codifica un insieme valido di triple e } \exists z < v((x, y, z) \in S_v)".$$

Essendo questo un predicato decidibile, è calcolabile anche la funzione

$$f(x, y) = \mu v R(x, y, v)$$

Ne consegue che la funzione di Ackermann è calcolabile tramite

$$\psi(x, y) = \mu z((x, y, z) \in S_{f(x, y)})$$

Capitolo 8

Loop Program

I loop program sono un sottoinsieme dei programmi URM.

8.1 Istruzioni dei loop program

L'istruzione di salto condizionato, viene sostituita con l'istruzione loop, che consiste in realtà in due keyword: $\text{Loop}(n)$, End . Somiglia un po' a un ciclo for, in quanto, sia k il valore del registro R_n , il programma presente tra le keyword viene ripetuto k volte.

8.1.1 Mappare Loop program in un programma URM

Algoritmo 12 *Loop program*

Loop(n)
P
End

Algoritmo 13 *Loop program in URM*

T(n, m*)
Z(m*+1)
[I_p] :
J(m* + 1, m*, q)
P
S(m*+1)
J(1, 1, p)

8.2 Gerarchia dei Loop program

Distinguiamo i loop program secondo il numero di cicli loop annidati. Usiamo quindi

- L_0 : insieme dei loop program che presentano sole istruzioni $Z(n), S(n), T(m, n)$.
- L_{k+1} : insieme dei loop program costituiti da $Z(n), S(n), T(m, n)$ e *Loop* contenenti $P \in L_k$.

È una notazione ricorsiva. Banalmente $L_k \subset L_{k+1}$ per $k \in \mathbb{N}$.

8.2.1 Gerarchia delle *funzioni* calcolate dai Loop program

Per ogni $n \geq 0$, indichiamo con $\mathcal{L}_n$ l'insieme delle funzioni **calcolate da programmi** in L_n .

Funzione di Ackermann n -esima Osserviamo che, data ψ_{n+1} funzione di Ackermann ennesima $\psi_{n+1} \in \mathcal{L}_{n+1} \setminus \mathcal{L}_n$.

8.3 Funzioni primitive ricorsive e i Loop Program

Indichiamo con $\mathcal{L} = \bigcup_{n=0}^{\infty} \mathcal{L}_n$, ossia l'insieme di tutti i loop program. Vogliamo dimostrare che

$$\mathcal{PR} = \bigcup_{n=0}^{\infty} \mathcal{L}_n = \mathcal{L}$$

La dimostrazione consisterà in

1. Dimostrare che le funzioni iniziali appartengono a $\mathcal{L}_0$.
2. Dimostrare che $\bigcup_{n=0}^{\infty} \mathcal{L}_n$ è chiuso rispetto a composizione e ricorsione primitiva, concludendo che $\mathcal{PR} \subseteq \bigcup_{n=0}^{\infty} \mathcal{L}_n$.
3. Dimostrare $\mathcal{PR} \supseteq \bigcup_{n=0}^{\infty} \mathcal{L}_n$ per induzione su n .

1. Dimostrazione funzioni iniziali Le funzioni iniziali sono in $\mathcal{L}_0$, e coincidono con le uniche istruzioni ammesse da $\mathcal{L}_0$. Triviale.

2.1. Dimostrazione composizione Siano $f(y_1, \dots, y_k), g_1(\vec{x}), \dots, g_k(\vec{x}) \in \mathcal{L}_p$. Sia $h(\vec{x}) = f(g_1(\vec{x}), \dots, g_k(\vec{x}))$. Il programma che calcola la composizione $h(\vec{x})$ è il già ben noto

```

T(1, m + 1)
⋮
T(n, m + n)
Rm+n+1 ← fG1(Rm+1, ..., Rm+n)
⋮
Rm+n+k ← fGk(Rm+1, ..., Rm+n)
R1 ← fF(Rm+n+1, ..., Rm+n+k)

```

e si trova in $\mathcal{L}_p$.

2.2. Dimostrazione ricorsione primitiva Lo stesso vale per la ricorsione primitiva. Infatti, se le funzioni delle equazioni ricorsive $f(\vec{x}), g(\vec{x}, y, z) \in \mathcal{L}_p$, allora la funzione $h(\vec{x}, y)$ definita per ricorsione primitiva si troverà in $\mathcal{L}_{p+1}$.

Di conseguenza, siano F e G che calcolano le funzioni f, g , allora $F, G \in \mathcal{L}_p$, mentre il programma che calcola $h(\vec{x}, y)$ è in $\mathcal{L}_{p+1}$, in quanto ottenuto da un loop che itera per y volte, ovviamente calcolabile.

Abbiamo pertanto dimostrato che $\bigcup_{n=0}^{\infty} \mathcal{L}_n$ è chiuso rispetto a composizione e ricorsione primitiva.

3.0.1. Notazione vettoriale Prima di dimostrare l'ultima inclusione della dimostrazione, introduciamo la seguente notazione: la seguente funzione

$$\vec{f}: \mathbb{N}^m \rightarrow \mathbb{N}^n$$

è una funzione vettoriale che prende in input elementi di tipo $\vec{x} \in \mathbb{N}^m$, e cui output è dato da $\vec{f}(\vec{x}) = (f_1(\vec{x}), \dots, f_n(\vec{x}))$. Ciascuna delle componenti di $\vec{f}$, ossia $f_i \forall i \in \{1, \dots, n\}$, è del tipo $f_i: \mathbb{N}^m \rightarrow \mathbb{N}$. È detta anche funzione vettoriale di tipo (m, n) . Una funzione vettoriale di tipo (m, n) si dice **calcolabile** se ciascuna delle sue componenti f_i è calcolabile.

3.0.2. Proprietà delle funzioni vettoriali

- Composizione: se due funzioni vettoriali $\vec{f}, \vec{g}$ rispettivamente di tipo (m, n) ed (n, p) sono primitive ricorsive, tale lo è la loro composizione $\vec{f} \circ \vec{g}$, che sarà di tipo (m, p) .

Capitolo 9

Altri approcci alla computabilità e tesi di Church

9.1 Altri approcci alla computabilità

Il modello URM è uno dei modelli più recenti usati nello studio della calcolabilità. Tra gli approcci alla calcolabilità, questi sono alcuni dei modelli più noti:

- Godel - Herbrand - Kleene: funzioni ricorsive generali.
- Church: λ -calcolo.
- Godel - Kleene: funzioni μ -ricorsive e funzioni ricorsive parziali. ★
- Turing: funzioni calcolabili mediante macchine di Turing. ★
- Post: funzioni definite mediante sistemi deduttivi canonici.
- Sheperdson-Sturgis: funzioni URM-calcolabili. ★

i modelli ★ saranno oggetti del nostro studio.

Risultato della teoria della calcolabilità Tutti i modelli precedentemente proposti caratterizzano la medesima classe di **funzioni calcolabili**, che abbiamo precedentemente denotato con $\mathcal{C}$, per cui non ha più senso dire $\mathcal{C}_{\text{URM}}$.

9.2 Funzioni ricorsive parziali (Godel - Kleene)

La classe $\mathcal{R}$ delle **funzioni ricorsive parziali** è la più piccola delle famiglie di funzioni parziali che

- Contiene le funzioni iniziali $\underline{0}, x + 1, \bigcup_i^n$.
- È chiusa rispetto a **composizione**, **ricorsione primitiva**, **minimalizzazione**.

Godel e Kleene hanno inizialmente portato la loro attenzione sulla classe $\mathcal{R}_0$, ossia delle **funzioni μ -ricorsive**, che coincide con $\mathcal{R}$ e ammette la minimalizzazione **solo se non consente funzioni parziali**.

$$\mathcal{R}_0 = \mathcal{R} \cap TOT$$

9.2.1 Tutte le funzioni ricorsive parziali sono calcolabili

$$\mathcal{R} = \mathcal{C}_{\text{URM}}$$

Dimostrazione Per quanto visto in precedenza, $\mathcal{R} \subseteq \mathcal{C}_{URM}$.

Viceversa, sia $f(\vec{x})$ una funzione URM-calcolabile da un programma $P = I_1, I_2, \dots, I_s$. Senza perdere di generalità, possiamo supporre P sia in forma standard, e che non ci siano istruzioni di salto che facciano passare all'istruzione subito successiva. Definiamo quindi un *passo* come l'esecuzione di un'istruzione.

Consideriamo adesso le seguenti le seguenti funzioni relative alla computazione di P .

$$c(\vec{x}, t) = \begin{cases} R_1 & \text{dopo } t \text{ passi della computazione } P(\vec{x}) \text{ se questa non si è interrotta.} \\ R_1 & \text{a fine computazione, se si è interrotta.} \end{cases}$$

$$j(\vec{x}, t) = \begin{cases} \text{numero della prossima istruzione} & \text{dopo } t \text{ passi di } P(\vec{x}) \text{ se questa non si è interrotta.} \\ 0 & \text{se si è interrotta.} \end{cases}$$

queste sono evidentemente funzioni totali. Osserviamo quindi che

$$f_P^{(n)}(\vec{x}) \downarrow \Leftrightarrow \mu t(j(\vec{x}, t))$$

NON STO CAPENDO NULLA

9.3 Macchine di Turing

Una macchina di Turing M è un dispositivo finito che effettua operazioni primitive su un nastro infinito. Il nastro infinito è suddiviso in caselle che possono contenere un simbolo tratto da una lista finita di simboli, detto **alfabeto della macchina**. s_0 è il simbolo "blank", o B .

M può effettuare operazioni di lettura e scrittura (tramite una testina), e spostarsi a sinistra o a destra sul nastro. Ha inoltre un'unità di controllo costituita da un insieme finito di quadruple.

Quadruple Ciascuna di queste quadruple è nella forma

$$q_i \ s_j \ \alpha \ q_l$$

con q_i stato della macchina, s_j carattere letto, $\alpha \in \{L, R, s_0, \dots, s_k\}$ e q_l nuovo stato. Ogni quadrupla definisce quindi quale azione effettuare (spostamento a sx, a dx o sovrascrittura di quale simbolo), quando la macchina legge un carattere in un determinato stato, e in quale stato transitare.

Convezione sul contare Negli usi che faremo, rappresenteremo il numero n con n caselle contenenti il simbolo 1. La funzione che conta il numero di occorrenze di 1 sul nastro è Turing-calcolabile.

9.3.1 Funzioni Turing calcolabili

Una funzione parziale si dice **Turing-calcolabile** se esiste una macchina di Turing che la calcola. La famiglia delle funzioni Turing-calcolabili si indica con $\mathcal{C}_{TUR}$.

9.3.2 Equivalenza tra URM e TM

Le funzioni ricorsive parziali $\mathcal{R}$ sono URM-calcolabili, ma anche Turing-calcolabili, ossia

$$\mathcal{C}_{URM} = \mathcal{R} = \mathcal{C}_{TUR}$$

e ciò è dimostrabile per **simulazione**.

9.4 Tesi di Church-Turing

La tesi di Church Turing è uno dei pilastri teorici dell'informatica moderna, e ha un ruolo fondamentale nella teoria della computabilità. Afferma che:

*La classe delle funzioni **intuitivamente** (umanamente) calcolabili coincide con la classe $\mathcal{C}_{URM}$, ossia quella delle funzioni URM-calcolabili. Lo stesso vale per le macchine di Turing, e qualsiasi altro sistema formale di computazione. È calcolabile tutto ciò che può essere calcolato da una macchina di Turing.*

A supporto di questa tesi, abbiamo che:

- La tesi di Church-Turing risulta essere il teorema fondamentale della calcolabilità, in quanto tutti i sistemi di calcolabilità sembrano convergere alla stessa classe di funzioni calcolabili.
- Si ha una collezione di funzioni per la quale esiste una dimostrazione di appartenenza a $\mathcal{C}_{URM} = \mathcal{C}_{TUM} = \mathcal{R}$.
- La possibilità di simulare effettivamente programmi URM.
- La mancanza di controesempi di funzioni effettivamente¹ calcolabili ma non URM-calcolabili.

9.4.1 La potenza della tesi di Church-Turing

Basandosi sulle prove precedentemente descritte, tutti i matematici hanno accettato la tesi di Church, la quale risulta essere uno strumento estremamente potente nelle dimostrazioni. Piuttosto che usare un modello teorico, come quello URM, o le macchine di Turing, possiamo procedere nel seguente modo.

Dimostrazione della calcolabilità per tesi di Church Desideriamo dimostrare la calcolabilità della funzione f . Definiamo (in linguaggio naturale, tramite diagrammi o schemi) un algoritmo, e forniamo una dimostrazione informale ma rigorosa relativa al fatto che l'algoritmo calcola effettivamente la funzione f . Appelliamoci poi alla tesi di Church e concludiamo immediatamente dicendo che f è URM-calcolabile.

Estratto dal libro (che personalmente adoro) riguardo la tesi di Church-Turing

Note that in using Church's thesis we are not proposing to abandon all thought of proof, as if Church's thesis is a magic wand which we can wave instead. A proof by Church's thesis will always involve proof that is careful, and sometimes complicated, although informal. [...]
Church's thesis not only keeps proofs shorter, but also prevents the main idea of a proof or construction from being obscured by a mass of technical details. It remains, however, an expression of faith or confidence. The validity of faith depends on the evidence that can be mustered. In the case of Church's thesis, there is the mathematical evidence already outlined. For the practised student there is the additional evidence of his own experience in translating informal algorithms into formal counterparts. For the beginner, our use of Church's thesis in subsequent chapters may call on his willingness to place confidence in the ability of others until self confidence is developed.

¹Nella teoria della calcolabilità, un algoritmo effettivo (o procedura effettiva) è un insieme finito di istruzioni precise, non ambigue e meccaniche che, se applicato a un problema, produce una soluzione corretta in un numero finito di passi. Deve essere eseguibile da un essere umano o macchina senza intuizione, garantendo lo stesso risultato.

Capitolo 10

Enumerazione delle funzioni calcolabili

In questo capitolo andremo a trattare l'enumerabilità dei programmi, e quindi dell'insieme delle funzioni calcolabili.

10.1 Enumerazione

Un insieme infinito X è enumerabile se esiste una biezione $f : X \rightarrow \mathbb{N}$. Se f ed f^{-1} sono anche **effettivamente calcolabili**, l'insieme X si dirà **effettivamente enumerabile**.

Nello specifico, **una enumerazione di un insieme X** è una suriezione $g : \mathbb{N} \rightarrow X$, (indicata con $X = \{x_0, x_1, x_2, \dots\}$). Quando g è iniettiva, diremo anche che è un'**enumerazione senza ripetizioni**.

10.1.1 Alcuni risultati tecnici

I seguenti insieme sono effettivamente enumerabili:

1. $\mathbb{N} \times \mathbb{N}$
2. $\mathbb{N}^+ \times \mathbb{N}^+ \times \mathbb{N}^+$
3. $\bigcup_{k>0} \mathbb{N}^k$

Dimostrazioni

1. Consideriamo la biezione $\pi : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$, definita da

$$\pi(m, n) =_{def} 2^m(2n + 1) - 1$$

e cui funzione inversa è data da

$$\pi^{-1}(x) = (\pi_1(x), \pi_2(x)) \quad \text{con} \quad \pi_1(x) =_{def} (x + 1), \quad \pi_2(x) =_{def} \frac{1}{2} \left(\frac{x + 1}{2^{\pi_1(x)}} - 1 \right)$$

2. Consideriamo la biezione

$$\xi : \mathbb{N}^+ \times \mathbb{N}^+ \times \mathbb{N}^+ \rightarrow \mathbb{N} : \xi(m, n, q) =_{def} \pi(\pi(m - 1, n - 1), q - 1)$$

$$\xi^{-1}(m, n, q) =_{def} (\pi_1(\pi_1(x)) + 1, \pi_2(\pi_1(x)) + 1, \pi_2(x) + 1)$$

le cui funzioni sono **effettivamente calcolabili**.

3. Si consideri la biezione $\tau : \bigcup_{k>0} \mathbb{N}^k \rightarrow \mathbb{N}$.

$$\tau(a_1, \dots, a_k) =_{def} 2^{a_1} + 2^{a_1+a_2+1} + \dots + 2^{a_1+a_2+\dots+a_k+k-1} - 1$$

questa è evidentemente una funzione calcolabile, ma per verificare che questa sia una biezione, andremo a sfruttare il fatto che ogni numero naturale ha una e una sola espressione binaria decimale. Noto x , possiamo trovare i numeri univoci $k \geq 1$ e i termini $0 \leq b_1 < b_2 < \dots < b_k$ tale che

$$x + 1 = 2^{b_1} + 2^{b_2} + \dots + 2^{b_k}$$

dove $b_i = a_1 + \dots + a_i + i - 1 \Rightarrow a_{i+1} = b_{i+1} - b_i - 1 \forall i : 1 \leq i < k$ (e $a_1 = b_1$). La funzione $\tau^{-1}(x)$ è quindi effettivamente calcolabile, confermando la biattività effettiva.

10.2 Enumerabilità delle funzioni calcolabili

Dimostriamo passo passo l'enumerabilità dei programmi, partendo dalla consapevolezza che, un programma qualsiasi, è una sequenza finita di istruzioni. Dimostriamo che l'insieme delle istruzioni è enumerabile.

Notazione

- $\mathcal{I}$: insieme di tutte le istruzioni URM.
- $\mathcal{P}$: insieme di tutti i programmi URM.

10.2.1 Enumerabilità delle istruzioni

Dimostriamo l'enumerabilità dell'insieme delle istruzioni $\mathcal{I}$, definendo una biezione $\beta : \mathcal{I} \rightarrow \mathbb{N}$.

- $\beta(Z(n)) = 4(n - 1)$
- $\beta(S(n)) = 4(n - 1) + 1$
- $\beta(T(m, n)) = 4\pi(m - 1, n - 1) + 2$
- $\beta(J(m, n, q)) = 4\xi(m, n, q) + 3$

la funzione inversa $\beta^{-1}(x)$, consiste nel trovare i valori $u, r : x = 4u + r$, con $r \in \{0, 1, 2, 3\}$. Otteniamo quindi

$$\beta^{-1}(x) = \begin{cases} Z(u + 1) & \text{se } r = 0 \\ S(u + 1) & \text{se } r = 1 \\ T(\pi_1(x) + 1, \pi_2(u) + 1) & \text{se } r = 2 \\ J(\xi^{-1}(u)) & \text{se } r = 3 \end{cases}$$

ricordiamo che $\xi^{-1}(u) = (m, n, q)$. Esistono quindi delle funzioni calcolabili per una biezione tra $\mathcal{I}, \mathbb{N}$, rendendo $\mathcal{I}$ enumerabile.

10.2.2 Enumerabilità dei programmi URM

Definiamo adesso una biezione $\gamma : \mathcal{P} \rightarrow \mathbb{N}$. Sia $P \in \mathcal{P}$. Se il programma $P = I_1, I_2, \dots, I_s$, poniamo $\gamma(P) = \tau(\beta(I_1), \beta(I_2), \dots, \beta(I_s))$. È facile verificare che γ, γ^{-1} sono effettivamente calcolabili, e che quindi $\mathcal{P}$ è effettivamente enumerabile.

10.2.3 Numero di Gödel

Dato un programma $P \in \mathcal{P}$, $\gamma(P)$ è detto **numero di Gödel** di P (o codice di P). Poniamo inoltre $P_n = \gamma^{-1}(n)$.

Esempio di calcolo di Numero di Gödel $P : T(1, 3), S(4), Z(6)$. Si ha:

- $\beta(T(1, 3)) = 4\pi(0, 2) + 2 = 4(2^0(2 \cdot 2 + 1) - 1) + 2 = 18$
- $\beta(S(4)) = 4 \cdot 3 + 1 = 13$
- $\beta(Z(6)) = 4 \cdot 5 = 20$

$$\begin{aligned} \gamma(P) &= 2^{18} + 2^{18+13+1} + 2^{18+13+20+2} - 1 \\ &= 2^{18} + 2^{32} + 2^{53} - 1 \\ &= 262144 + 4294967296 + 9007199254740992 - 1 \\ &= 9007203549970431 \end{aligned}$$

10.3 Enumerabilità delle funzioni

Forniamo delle notazioni utili

- $\phi_a^{(n)}$: la funzione n -aria calcolata da P_a , ossia $f_{P_n}^{(n)}$.
- $W_a^{(n)}$: il dominio di $\phi_a^{(n)}$.
- $E_a^{(n)}$: l'immagine di $\phi_a^{(n)}$.

Se f è una funzione **unaria calcolabile**, $f = \phi_a$ per qualche $a \in \mathbb{N}$, con $a = \gamma(P)$. Diremo che a è un indice per f . Siccome esistono infiniti programmi che calcolano la stessa funzione, la sequenza $\alpha_0, \alpha_1, \alpha_2, \dots$ è una sequenza **con ripetizioni** che contiene tutte le funzioni unarie calcolabili. Generalizzando anche le arietà, la sequenza $\alpha_0^{(n)}, \alpha_1^{(n)}, \alpha_2^{(n)}, \dots$ è una sequenza **con ripetizioni** che contiene tutte le funzioni n -arie calcolabili.

Teorema $C_n = \{\phi_a^{(n)} : a \in \mathbb{N}\}$, ossia l'insieme delle funzioni calcolabili, è enumerabile.

Dimostrazione

10.4 Diagonalizzazione di Cantor

Data un'enumerazione x_0, x_1, x_2 di funzioni parziali da $\mathbb{N}$ in $\mathbb{N}$, il **metodo diagonale di Cantor** consente di costruire una funzione x da $\mathbb{N}$ in $\mathbb{N}$, diversa da tutte le funzioni x_i dell'enumerazione

	0	1	2	3	...
χ_0	$\chi_0(0)$	$\chi_0(1)$	$\chi_0(2)$	$\chi_0(3)$	...
χ_1	$\chi_1(0)$	$\chi_1(1)$	$\chi_1(2)$	$\chi_1(3)$	...
χ_2	$\chi_2(0)$	$\chi_2(1)$	$\chi_2(2)$	$\chi_2(3)$	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

10.4.1 Teorema sull'esistenza di una funzione totale, unitaria e non calcolabile

Esiste una funzione totale, unitaria e non calcolabile.

$$f(n) = \begin{cases} \phi_n(n) + 1 & \text{se } \phi_n(n) \downarrow \\ 0 & \text{se } \phi_n(n) \uparrow \end{cases}$$

Capitolo 11

Decidibilità

11.1 Teorema *s-m-n* o di parametrizzazione

Sia $f(x, y)$ una funzione calcolabile, e dato $a \in \mathbb{N}$, poniamo

$$f_a(y) =_{def} f(a, y)$$

f_a è calcolabile, quindi $f_a = \varphi_{e_a}$

Capitolo 12

Programmi universali

12.1 Programma universale

Si consideri la funzione $\psi(x, y) =_{def} \varphi_x(y)$. Questa funzione è universale, in quanto, ponendo $g_m(y) =_d ef \psi(m, y)$, la sequenza di $g_0, g_1, g_2, \dots$ contiene tutte le funzioni unarie calcolabili $\varphi_0^{(1)}, \varphi_1^{(1)}, \varphi_2^{(1)}, \dots$.

12.1.1 Calcolabilità della funzione universale

Per ogni arità $n \geq 1$, la funzione universale $\psi_U^{(n)}$ è calcolabile.

Dimostrazione

12.2 Esistenza di funzione *totale* calcolabile non primitiva ricorsiva

Esiste una funzione *totale* calcolabile non primitiva ricorsiva

Dimostrazione

12.3 Operazioni effettive su funzioni calcolabili

Ci chiediamo se sia possibile manipolare in maniera effettiva funzioni o insiemi infiniti (ricorsivamente enumerabili¹): in alcuni casi, questo è possibile, tramite applicazione del teorema di parametrizzazione!

12.3.1 Esempio 1

Date φ_x, φ_y , calcolare un indice di $\varphi_x \varphi_y$.

$$\begin{aligned}\varphi_x(z) \cdot \varphi_y(z) &= \psi_U(y_1, z) \cdot \psi_U(y, z) \\ &= \varphi_e^{(3)}(x, y, z) \\ &= \varphi_{S_1^2(e, x, y)}(z) \\ &= \varphi_{k(x, y)}(z) \quad (\text{con } k(x, y) := S_1^2(e, x, y) \text{ totale calcolabile}) \\ \implies \varphi_x \cdot \varphi_y &= \varphi_{k(x, y)}\end{aligned}$$

12.3.2 Esempio 2

Esiste una funzione calcolabile totale $g(x)$ tale che $(\varphi_x)^2 = \varphi_{g(x)}$.

¹Gli insiemi ricorsivamente enumerabili sono collezioni di elementi (solitamente numeri naturali) per i quali esiste un algoritmo capace di elencarne i membri o di riconoscere quelli validi in tempo finito

$$\begin{aligned}
\varphi_x(z) \cdot \varphi_y(z) &= \psi_U(y_1, z) \cdot \psi_U(y, z) \\
&= \varphi_e^{(3)}(x, y, z) \\
&= \varphi_{S_1^2(e, x, y)}(z) \\
&= \varphi_{k(x, y)}(z) \quad (\text{con } k(x, y) := S_1^2(e, x, y) \text{ totale calcolabile}) \\
\implies \quad \varphi_x \cdot \varphi_y &= \varphi_{k(x, y)}
\end{aligned}$$

Capitolo 13

???

13.1 Problemi indecidibili della teoria della calcolabilità

13.1.1 Halting problem

Il problema " $x \in W_x$ " è indecidibile. Si dimostra verificando che la funzione

Definizioni, sinonimi, chiarimenti

13.2 Sinonimi

- *Calcolabilità e computabilità.*
- *Composizione e sostituzione.*